

JAVA INTERVIEW QUESTION BANK

780 Questions • 13 Topics • 60 Questions Per Topic

Interview-style “Can we...?” / “Is it possible...?” revision sheet

1. Methods — 60 Questions

#	Question	Answer	Main Reason
1	Can a method be declared inside a class?	✓ Yes	Methods are declared inside classes or interfaces
2	Can a method have parameters?	✓ Yes	Parameters provide input to a method
3	Can a method have no parameters?	✓ Yes	Parameters are optional
4	Can a method return a value?	✓ Yes	Use a suitable non-void return type
5	Can a method return nothing?	✓ Yes	Use void
6	Can a method be public?	✓ Yes	public allows broad access
7	Can a method be private?	✓ Yes	private restricts access to its class
8	Can a method be protected?	✓ Yes	protected allows package and subclass access
9	Can a method have default access?	✓ Yes	No access modifier gives package-private access
10	Can a method be static?	✓ Yes	static makes it a class member
11	Can a method be final?	✓ Yes	final prevents overriding
12	Can a method be abstract?	✓ Yes	An abstract method has no implementation
13	Can a method be synchronized?	✓ Yes	synchronized controls concurrent access
14	Can a method be native?	✓ Yes	native methods can use non-Java implementations
15	Can a method be overloaded?	✓ Yes	Parameter lists can differ
16	Can a static method be overloaded?	✓ Yes	Static methods can have different parameter lists
17	Can a non-static method be overloaded?	✓ Yes	Instance methods can have different parameter lists
18	Can static and non-static methods be overloaded?	✓ Yes	Their parameter lists must differ
19	Can methods be overloaded by return type only?	✗ No	Return type alone is not part of the method signature
20	Can methods be overloaded by access modifier only?	✗ No	Access modifier does not distinguish overloads
21	Can methods be overloaded by changing parameter type?	✓ Yes	Different parameter types create overloads
22	Can methods be overloaded by changing parameter count?	✓ Yes	Different parameter counts create overloads
23	Can methods be overloaded by changing parameter order?	✓ Yes	Different parameter sequences can create overloads
24	Can a static method directly access an instance variable?	✗ No	It needs an object reference
25	Can a static method access a static variable directly?	✓ Yes	Both belong to the class
26	Can an instance method access a static variable?	✓ Yes	Instance methods can access class members
27	Can an instance method access an instance variable?	✓ Yes	It runs with an object context
28	Can a static method directly call an instance method?	✗ No	There is no implicit object context
29	Can an instance method call a static method?	✓ Yes	Static methods belong to the class

30	Can a method call another method?	✔ Yes	Methods can call accessible methods
31	Can a method call itself?	✔ Yes	This is called recursion
32	Can a method have a local variable?	✔ Yes	Local variables can be declared inside methods
33	Can a method contain if statements?	✔ Yes	Methods can contain normal Java statements
34	Can a method contain loops?	✔ Yes	Loops can be used inside methods
35	Can a method contain another method declaration directly?	✘ No	Java does not support nested named method declarations
36	Can a method have the same name as its class?	✔ Yes*	It is allowed, but without a return-type rule it is a method, not a constructor
37	Can a method have a return type void?	✔ Yes	void means no value is returned
38	Can a void method use return?	✔ Yes*	It may use return; to exit without a value
39	Can a non-void method use return?	✔ Yes	It must return a compatible value on required paths
40	Can a method return an object?	✔ Yes	Reference types can be return types
41	Can a method return an array?	✔ Yes	Arrays are valid return types
42	Can a method return another method?	✘ No	Java methods are not first-class method values
43	Can a method accept an array parameter?	✔ Yes	Arrays can be method parameters
44	Can a method accept an object parameter?	✔ Yes	Object references can be parameters
45	Can a method be called using an object?	✔ Yes*	Instance methods are normally called through objects
46	Can a static method be called using an object?	✔ Yes*	It is allowed but class-name access is preferred
47	Can a method be called using the class name?	✔ Yes*	Static methods can be called through the class
48	Can a private method be called within the same class?	✔ Yes	The declaring class has access
49	Can a private method be directly called from another class?	✘ No	private restricts direct access
50	Can a method be both static and final?	✔ Yes	These modifiers do not conflict
51	Can a method be both private and final?	✔ Yes	These modifiers do not conflict
52	Can a method be abstract and final?	✘ No	abstract needs overriding but final prevents it
53	Can a method be abstract and static?	✘ No	Static methods are not overridden
54	Can a method be abstract and private?	✘ No	Private methods cannot be overridden
55	Can a method be synchronized and static?	✔ Yes	Both modifiers can be used together
56	Can a method be synchronized and final?	✔ Yes	These modifiers can coexist
57	Can a method be native and abstract?	✘ No	Both require incompatible implementation rules
58	Can a method be native and final?	✔ Yes	These modifiers can coexist
59	Can a method be overloaded with varargs?	✔ Yes	Varargs can participate in overloads
60	Can a method be generic?	✔ Yes	Methods can declare type parameters

2. Method Overloading — 60 Questions

#	Question	Answer	Main Reason
---	----------	--------	-------------

1	Can method overloading exist in the same class?	✓ Yes	Overloading uses different parameter lists
2	Can overloaded methods have the same name?	✓ Yes	Same name is required
3	Must overloaded methods have different parameters?	✓ Yes	Parameters distinguish overloads
4	Can overloaded methods differ by parameter count?	✓ Yes	Different counts are valid
5	Can overloaded methods differ by parameter type?	✓ Yes	Different types are valid
6	Can overloaded methods differ by parameter order?	✓ Yes	Different type order can distinguish them
7	Can overloaded methods differ only by return type?	✗ No	Return type alone cannot distinguish calls
8	Can overloaded methods differ only by access modifier?	✗ No	Access modifier is not enough
9	Can static methods be overloaded?	✓ Yes	Static methods can have overloads
10	Can instance methods be overloaded?	✓ Yes	Instance methods can have overloads
11	Can static and instance methods be overloads?	✓ Yes	Parameter lists must differ
12	Can private methods be overloaded?	✓ Yes	Private methods can be overloaded in their class
13	Can final methods be overloaded?	✓ Yes	final prevents overriding, not overloading
14	Can abstract methods be overloaded?	✓ Yes*	An abstract class/interface can declare different abstract signatures
15	Can constructors be overloaded?	✓ Yes	Constructors can have different parameters
16	Can main() be overloaded?	✓ Yes	main is still a method
17	Can overloaded methods have different return types with different parameters?	✓ Yes	Different parameters make the overload distinct
18	Can overloaded methods have the same return type?	✓ Yes	Return type may remain the same
19	Can overloaded methods have different access modifiers?	✓ Yes*	Access may differ if parameters also distinguish them
20	Can overloaded methods be inherited?	✓ Yes	Accessible overloads can be inherited
21	Can a child add an overload of a parent method?	✓ Yes	A different parameter list creates an overload
22	Does overloading happen at compile time?	✓ Yes	The compiler selects the overload
23	Is overloading compile-time polymorphism?	✓ Yes	Method selection is compile-time
24	Does overloading require inheritance?	✗ No	It can occur within one class
25	Can overloaded methods be in different classes?	✓ Yes*	A child can add an overload to an inherited method
26	Can a method be overloaded using int and double?	✓ Yes	Parameter types differ
27	Can a method be overloaded using int and int,int?	✓ Yes	Parameter counts differ
28	Can a method be overloaded using String and Object?	✓ Yes	Parameter types differ
29	Can a method be overloaded using int[] and int... ?	✗ No	These parameter forms have the same effective array signature
30	Can varargs participate in overloading?	✓ Yes	Varargs is part of parameter declaration
31	Can boxing affect overload selection?	✓ Yes	Conversions influence which overload is selected
32	Can widening affect overload selection?	✓ Yes	Primitive widening is considered
33	Can unboxing affect overload selection?	✓ Yes	Unboxing is a possible conversion
34	Can widening be preferred over boxing in overload resolution?	✓ Yes*	Method invocation rules prefer certain conversions

35	Can two overloads have ambiguous calls?	✓ Yes	Some conversions can make a call ambiguous
36	Can null cause an overloaded call to be ambiguous?	✓ Yes	Null may match multiple unrelated reference types
37	Can overloaded methods be private?	✓ Yes	Access does not prevent overloading
38	Can overloaded methods be static?	✓ Yes	Static methods support overloading
39	Can overloaded methods be final?	✓ Yes	final affects overriding only
40	Can overloaded methods be synchronized?	✓ Yes	synchronized does not prevent overloading
41	Can overloaded methods be generic?	✓ Yes	Generic methods can be overloaded
42	Can generic methods be overloaded by different type bounds only?	✗ No*	Erasure/signature rules may make declarations clash
43	Can methods be overloaded using generic parameter types?	✓ Yes*	They can if the resulting signatures are distinct
44	Can methods be overloaded by changing generic return type only?	✗ No	Return type alone is insufficient
45	Can methods be overloaded by changing exception list only?	✗ No	Throws clauses do not form the signature
46	Can methods be overloaded by changing static to non-static only?	✗ No*	If parameters are identical, the declaration conflicts
47	Can overloaded methods have different throws clauses?	✓ Yes*	Throws clauses may differ when signatures differ or where otherwise legal
48	Can an overloaded method call another overload using this()?	✗ No*	this() is specifically a constructor invocation
49	Can one overloaded method call another method overload directly?	✓ Yes	Normal method calls can select another overload
50	Can overloading reduce code duplication?	✓ Yes	Related operations can share one name
51	Can overloading improve readability?	✓ Yes	One meaningful method name can represent related operations
52	Can overloaded methods have different implementations?	✓ Yes	Each overload can have its own body
53	Can overloaded methods have different parameter names?	✓ Yes	Parameter names do not determine overloads
54	Can changing only parameter names create overloads?	✗ No	Parameter names are not part of the signature
55	Can changing only whitespace create overloads?	✗ No	Whitespace has no effect on signatures
56	Can changing only comments create overloads?	✗ No	Comments do not affect compilation
57	Can overloaded methods be selected based on runtime object type?	✗ No	Overload selection is primarily compile-time
58	Can a cast change which overload is selected?	✓ Yes	The compile-time type can change overload resolution
59	Can overloaded methods have the same parameter names?	✗ No*	If the parameter types/count are identical, the declarations conflict
60	Can changing only parameter names create an overload?	✗ No	Parameter names are not part of the signature

3. Method Overriding — 60 Questions

#	Question	Answer	Main Reason
1	Can a non-static instance method be overridden?	✓ Yes	Instance methods support overriding
2	Can a static method be overridden?	✗ No	Static methods are hidden
3	Can a private method be overridden?	✗ No	Private methods are not inherited
4	Can a final method be overridden?	✗ No	final prevents overriding

5	Can an abstract method be overridden?	✓ Yes	A concrete child provides its implementation
6	Must an overriding method have the same name?	✓ Yes	The method signature must match
7	Must an overriding method have the same parameters?	✓ Yes	Same parameter list is required
8	Can an overriding method change parameter types?	✗ No	That becomes overloading
9	Can an overriding method change only return type?	✗ No*	Return must be same or covariant
10	Can an overriding method use a covariant return type?	✓ Yes	The return type may be a subtype
11	Can an overriding method have wider access?	✓ Yes	Access can be increased
12	Can an overriding method have narrower access?	✗ No	Access cannot be reduced
13	Can an overriding method throw a broader checked exception?	✗ No	Checked exceptions cannot be broadened
14	Can an overriding method throw a narrower checked exception?	✓ Yes	The restriction may be reduced
15	Can an overriding method throw no checked exception?	✓ Yes	It may throw fewer checked exceptions
16	Can an overriding method throw unchecked exceptions?	✓ Yes	Unchecked exceptions are not restricted this way
17	Can a child override a parent method with @Override?	✓ Yes	@Override verifies overriding
18	Is @Override mandatory for overriding?	✗ No	It is recommended but not required
19	Can a parent reference call an overridden child method?	✓ Yes	Runtime dispatch selects the child implementation
20	Is method overriding runtime polymorphism?	✓ Yes	Selection occurs at runtime
21	Can overriding happen without inheritance?	✗ No	There must be a parent-child relationship
22	Can an interface method be implemented by a class?	✓ Yes	The class provides the implementation
23	Can an abstract class method be overridden by a child?	✓ Yes	The child can implement it
24	Can a constructor be overridden?	✗ No	Constructors are not inherited
25	Can a private method have the same signature in a child?	✓ Yes*	It is a new method, not an override
26	Can a static method with the same signature be hidden?	✓ Yes	Static methods are hidden
27	Can a final class contain methods that are overridden?	✗ No*	The class cannot have subclasses
28	Can a final method be overloaded?	✓ Yes	final prevents overriding, not overloading
29	Can an overriding method be final?	✓ Yes	The child can prevent further overriding
30	Can an overriding method be static when parent is instance?	✗ No	Static and instance methods cannot override each other
31	Can an overriding method be instance when parent is static?	✗ No	Static methods are hidden, not overridden
32	Can an overriding method be synchronized?	✓ Yes	synchronized may be added
33	Can an overriding method remove synchronized?	✓ Yes	synchronized is not required for overriding
34	Can an overriding method be native?	✓ Yes*	Subject to Java declaration rules
35	Can an overriding method be abstract?	✓ Yes*	An abstract intermediate class may keep it abstract
36	Can a child override a protected parent method as public?	✓ Yes	Access can be widened
37	Can a child override a public method as protected?	✗ No	Access would be reduced
38	Can a child override a protected method as private?	✗ No	Access would be reduced
39	Can super call the parent implementation?	✓ Yes	super.method() selects the parent implementation
40	Can this call the overridden method?	✓ Yes	this refers to the current object

41	Can a parent reference call a child-only method?	✗ No	Reference type controls accessible members
42	Can casting a parent reference expose a child method?	✓ Yes*	A valid downcast can expose child members
43	Can overriding change the method's parameter names?	✓ Yes	Parameter names do not matter
44	Can overriding change the method body?	✓ Yes	The child supplies different behavior
45	Can overriding change the method's access from protected to public?	✓ Yes	Wider access is allowed
46	Can overriding change return type from Number to Integer?	✓ Yes	Integer is a subtype of Number
47	Can overriding change return type from Integer to Number?	✗ No	That is not covariant
48	Can overriding use an unrelated return type?	✗ No	Return types must be compatible
49	Can an interface default method be overridden?	✓ Yes	A class or subinterface can provide its own implementation
50	Can an interface static method be overridden?	✗ No	Interface static methods are not inherited for overriding
51	Can a default interface method be overridden as public?	✓ Yes	Implementation must not reduce access
52	Can two parent interfaces provide conflicting default methods?	✗ No*	The implementing class must resolve the conflict
53	Can overriding provide different behavior for different child classes?	✓ Yes	This is a key use of runtime polymorphism
54	Can overriding be used to specialize inherited behavior?	✓ Yes	The child can customize the inherited method
55	Can overriding occur at compile time?	✗ No	Dynamic dispatch occurs at runtime
56	Can a method be both overridden and overloaded in a child?	✓ Yes	A child can override one signature and add another
57	Can an overridden method be called with super?	✓ Yes	super bypasses the dynamic override for that call
58	Can overriding support runtime method selection?	✓ Yes	The actual object determines the implementation
59	Can overriding preserve the same access modifier?	✓ Yes	The access level may remain unchanged
60	Can overriding preserve the same return type?	✓ Yes	The return type may be exactly the same

4. Polymorphism — 60 Questions

#	Question	Answer	Main Reason
1	Is polymorphism an OOP concept?	✓ Yes	It allows one interface/name to represent different behavior
2	Is method overloading compile-time polymorphism?	✓ Yes	The compiler selects an overload
3	Is method overriding runtime polymorphism?	✓ Yes	The JVM selects the overridden implementation
4	Can a parent reference hold a child object?	✓ Yes	This is upcasting
5	Can a child reference directly hold a parent object?	✗ No	A parent object is not necessarily a child
6	Can a parent reference call an overridden child method?	✓ Yes	Runtime dispatch selects the child method
7	Can a parent reference call a child-specific method directly?	✗ No	Reference type controls accessible members
8	Can casting expose a child-specific method?	✓ Yes*	A valid downcast is required
9	Can downcasting be unsafe?	✓ Yes	An invalid cast can cause ClassCastException
10	Can instanceof help before downcasting?	✓ Yes	It can check runtime type
11	Can polymorphism reduce coupling?	✓ Yes	Code can depend on parent types/interfaces

12	Can an interface reference hold an implementing object?	✓ Yes	Interfaces are reference types
13	Can an abstract class reference hold a child object?	✓ Yes	Upcasting is allowed
14	Can an interface reference call an overridden implementation?	✓ Yes	Runtime dispatch selects the implementation
15	Can a static method demonstrate runtime polymorphism?	✗ No	Static methods are resolved by type and hidden
16	Can a private method demonstrate overriding polymorphism?	✗ No	Private methods are not overridden
17	Can a final method be overridden for polymorphism?	✗ No	final prevents overriding
18	Can constructors provide runtime polymorphism?	✗ No	Constructors are not overridden
19	Can overloaded constructors show compile-time polymorphism?	✓ Yes*	Constructor overloading is compile-time selection
20	Can a method call be polymorphic?	✓ Yes	Overridden instance methods are dynamically dispatched
21	Can a variable reference be polymorphic?	✓ Yes	A reference can point to different subtype objects
22	Can the same method call produce different results for different child objects?	✓ Yes	Overriding enables different implementations
23	Can polymorphism work with interfaces?	✓ Yes	Interfaces support multiple implementations
24	Can polymorphism work with abstract classes?	✓ Yes	Abstract references can point to concrete children
25	Can polymorphism work without inheritance?	✗ No*	Subtype polymorphism needs a compatible type relationship
26	Can method overloading depend on runtime type?	✗ No	Overload selection uses compile-time types
27	Can method overriding depend on runtime type?	✓ Yes	The actual object type determines implementation
28	Can a parent array store different child objects?	✓ Yes	Subtypes can be stored in a parent-type array
29	Can an interface array store multiple implementations?	✓ Yes	All implementing objects share the interface type
30	Can polymorphism help write generic code?	✓ Yes	Common parent/interface types can be used
31	Can a method accept a parent type and receive a child object?	✓ Yes	Upcasting allows it
32	Can a method accept an interface and receive different implementations?	✓ Yes	Implementations are assignable to the interface
33	Can overriding change behavior without changing the call?	✓ Yes	The same call can dispatch differently
34	Can overloading change behavior based on argument types?	✓ Yes	Different overloads can execute
35	Can polymorphism be achieved through method overriding?	✓ Yes	This is runtime polymorphism
36	Can polymorphism be achieved through method overloading?	✓ Yes	This is compile-time polymorphism
37	Can a parent reference access parent fields polymorphically?	✗ No*	Fields are hidden, not dynamically overridden
38	Can static methods be dynamically dispatched?	✗ No	Static method calls are not virtual
39	Can instance methods be dynamically dispatched?	✓ Yes	Normal instance methods support dynamic dispatch
40	Can an overridden method call another overridden method?	✓ Yes	Dynamic dispatch can continue through the call
41	Can super be used to bypass an override?	✓ Yes	super selects the parent implementation
42	Can this refer to the runtime object?	✓ Yes	this refers to the current object
43	Can polymorphism support extensibility?	✓ Yes	New implementations can fit common contracts
44	Can a parent method reference a child implementation?	✓ Yes	Runtime dispatch provides this behavior
45	Can polymorphism be used in collections?	✓ Yes	Collections can store common parent/interface types
46	Can ArrayList<Parent> store Child objects?	✓ Yes	Child is assignable to Parent

47	Can generic type invariance prevent some polymorphic assignments?	✓ Yes	Java generics are generally invariant
48	Can wildcard generics support subtype flexibility?	✓ Yes	Wildcards allow controlled variance
49	Can an abstract method participate in runtime polymorphism?	✓ Yes	Concrete children implement it
50	Can default interface methods participate in polymorphism?	✓ Yes	Implementations can override them
51	Can multiple child classes override the same parent method differently?	✓ Yes	Each child can provide its own implementation
52	Can polymorphism improve maintainability?	✓ Yes	Behavior can be changed behind a common contract
53	Can polymorphism mean one object has many forms?	✓ Yes	An object can be viewed through different compatible types
54	Can upcasting happen implicitly?	✓ Yes	A child-to-parent conversion is widening
55	Does downcasting happen implicitly?	✗ No	An explicit cast is normally required
56	Can invalid downcasting cause an exception?	✓ Yes	ClassCastException can occur
57	Can polymorphism be used in dependency injection?	✓ Yes	Interfaces/parents allow interchangeable implementations
58	Can a parent-type parameter accept different child objects?	✓ Yes	Subtypes can be passed through the parent type
59	Can an interface parameter accept different implementations?	✓ Yes	Implementations are compatible with the interface
60	Can polymorphism be used with abstract methods?	✓ Yes	Concrete subclasses provide different implementations

5. Constructors — 60 Questions

#	Question	Answer	Main Reason
1	Is a constructor used to initialize an object?	✓ Yes	It runs during object creation
2	Is a constructor a method?	✗ No	It has constructor-specific rules
3	Must a constructor have the class name?	✓ Yes	Constructor name must match the class
4	Can a constructor have a return type?	✗ No	Constructors have no return type
5	Can a constructor be void?	✗ No	void would make it a method
6	Can a constructor be overloaded?	✓ Yes	Different parameter lists are allowed
7	Can a constructor be overridden?	✗ No	Constructors are not inherited
8	Can a constructor be inherited?	✗ No	Constructors belong to their class
9	Can a constructor be static?	✗ No	Constructors initialize instances
10	Can a constructor be final?	✗ No	Constructors are not overridden
11	Can a constructor be abstract?	✗ No	Constructors cannot be incomplete methods
12	Can a constructor be private?	✓ Yes	Useful for controlled creation
13	Can a constructor be public?	✓ Yes	Allows accessible object creation
14	Can a constructor be protected?	✓ Yes	Useful for package/subclass construction
15	Can a constructor have default/package access?	✓ Yes	No access modifier is allowed
16	Can a constructor call another constructor in the same class?	✓ Yes	Use this()
17	Can a constructor call a parent constructor?	✓ Yes	Use super()

18	Can this() and super() both be first statements?	✗ No	Only one explicit constructor invocation is allowed
19	Must this() or super() be the first statement?	✓ Yes	Constructor invocation must be first
20	Does Java insert super() automatically?	✓ Yes*	If no explicit constructor invocation is written
21	Does Java provide a default constructor automatically?	✓ Yes*	Only when no constructor is declared
22	Does declaring any constructor stop automatic default construction?	✓ Yes	The compiler no longer adds a default constructor
23	Can a parameterized constructor be declared?	✓ Yes	Constructors can accept parameters
24	Can a constructor have multiple parameters?	✓ Yes	Normal parameter rules apply
25	Can a constructor have no parameters?	✓ Yes	A no-argument constructor is valid
26	Can a constructor initialize instance variables?	✓ Yes	It can assign object state
27	Can a constructor initialize static variables?	✓ Yes*	It can access/assign static members
28	Can a constructor contain statements?	✓ Yes	Constructors have a body
29	Can a constructor contain if statements?	✓ Yes	Normal statements are allowed
30	Can a constructor contain loops?	✓ Yes	Loops are allowed
31	Can a constructor return a value?	✗ No	Constructors do not return values
32	Can a constructor use return;?	✗ No*	A constructor cannot use return to return a value
33	Can a constructor throw an exception?	✓ Yes	Constructors can declare or throw exceptions
34	Can a constructor be synchronized?	✗ No	Constructors cannot use synchronized
35	Can a constructor be native?	✗ No	Constructors cannot be native
36	Can a constructor be called like a normal method?	✗ No	It is invoked through object creation or chaining
37	Can a constructor be called using an object after creation?	✗ No	Constructors are not normal methods
38	Can a constructor call another overloaded constructor?	✓ Yes	Constructor chaining is supported
39	Can constructor chaining use this()?	✓ Yes	this() selects another constructor in the same class
40	Can constructor chaining use super()?	✓ Yes	super() selects a parent constructor
41	Can this() call a parent constructor?	✗ No	this() targets the same class
42	Can super() call a constructor in the same class?	✗ No	super() targets the parent class
43	Can a child constructor execute parent initialization first?	✓ Yes	Parent construction occurs before child body
44	Can an abstract class have a constructor?	✓ Yes	Its constructor initializes the abstract-class portion
45	Can an interface have a constructor?	✗ No	Interfaces are not instantiated
46	Can a final class have a constructor?	✓ Yes	final only prevents inheritance
47	Can a constructor be overloaded with different parameter types?	✓ Yes	Parameter types can differ
48	Can constructors have the same parameter list?	✗ No	That would duplicate a constructor signature
49	Can constructors differ only by return type?	✗ No	Constructors have no return type
50	Can constructors be overloaded with varargs?	✓ Yes	Varargs constructors are allowed
51	Can a constructor call a private constructor?	✓ Yes	Access rules permit it within the class
52	Can a subclass directly call a private parent constructor?	✗ No	Private constructors are inaccessible to subclasses
53	Can a parent constructor be called using super(arguments)?	✓ Yes	Arguments select the matching parent constructor

54	Can a constructor be used to enforce validation?	✓ Yes	It can validate initial state
55	Can constructor overloading support different initialization options?	✓ Yes	Different constructors can initialize differently
56	Can constructors be inherited through interfaces?	✗ No	Interfaces have no constructors
57	Can object creation fail because a constructor throws?	✓ Yes	An exception can prevent successful construction
58	Can a constructor access instance fields?	✓ Yes	The object is being initialized
59	Can a constructor call an instance method?	✓ Yes*	It is allowed, though calling overridable methods during construction can be risky
60	Can constructors participate in polymorphic overriding?	✗ No	They cannot be overridden

6. Inheritance — 60 Questions

#	Question	Answer	Main Reason
1	Can one class inherit another class?	✓ Yes	Use extends
2	Can a class extend one class?	✓ Yes	Java supports single class inheritance
3	Can a class extend multiple classes?	✗ No	Java does not support multiple class inheritance
4	Can a class implement multiple interfaces?	✓ Yes	Multiple interfaces are supported
5	Can an interface extend multiple interfaces?	✓ Yes	Multiple interface inheritance is supported
6	Can an interface extend a class?	✗ No	Interfaces extend interfaces
7	Can a class implement an interface?	✓ Yes	Use implements
8	Can a class extend an interface?	✗ No	A class implements an interface
9	Can an interface be implemented by a class?	✓ Yes	The class supplies required implementations
10	Are constructors inherited?	✗ No	Constructors belong to their class
11	Are private methods inherited as accessible methods?	✗ No	Private members are not accessible in the child
12	Are public methods inherited?	✓ Yes	Public members are inherited subject to rules
13	Are protected methods inherited?	✓ Yes	Protected members support subclass access
14	Can a child directly access a private parent field?	✗ No	private restricts direct access
15	Can a child access a protected parent member?	✓ Yes	protected permits subclass access
16	Can a child access a public parent member?	✓ Yes	public is accessible
17	Can a child override a final parent method?	✗ No	final prevents overriding
18	Can a child override a private parent method?	✗ No	It is not inherited for overriding
19	Can a child override a static parent method?	✗ No	Static methods are hidden
20	Can a child inherit an abstract method?	✓ Yes	It must implement it or remain abstract
21	Can a final class be inherited?	✗ No	final prevents extension
22	Can an abstract class be inherited?	✓ Yes	Abstract classes are designed for inheritance
23	Can an abstract class have a constructor?	✓ Yes	It initializes the parent portion
24	Can an abstract class have concrete methods?	✓ Yes	It can contain implemented methods

25	Can an abstract class have fields?	✓ Yes	It can have instance and static fields
26	Can a child class call a parent constructor?	✓ Yes	Use super()
27	Can a child call a parent method using super?	✓ Yes	super selects the parent implementation
28	Can a child access a parent public static variable?	✓ Yes	Static members can be inherited/accessed
29	Can a child access a parent private static variable directly?	✗ No	private access is restricted
30	Can Java support single inheritance?	✓ Yes	One direct class parent is allowed
31	Can Java support multilevel inheritance?	✓ Yes	A class can extend a child class
32	Can Java support hierarchical inheritance?	✓ Yes	Multiple children can extend one parent
33	Can Java support multiple inheritance with classes?	✗ No	It avoids class multiple-inheritance ambiguity
34	Can Java achieve multiple inheritance through interfaces?	✓ Yes	A class can implement multiple interfaces
35	Can a class extend a class and implement interfaces?	✓ Yes	One class plus multiple interfaces is allowed
36	Can an interface extend more than one interface?	✓ Yes	Interfaces support multiple inheritance
37	Can a class inherit methods and fields from a parent?	✓ Yes*	Accessible members can be inherited
38	Can a child access inherited members through this?	✓ Yes	this refers to the current child object
39	Can a child access parent members through super?	✓ Yes	super refers to the parent portion
40	Can a child replace inherited method behavior?	✓ Yes	It can override non-final instance methods
41	Can a child add new methods?	✓ Yes	A subclass can add its own behavior
42	Can a child reduce visibility of an overridden method?	✗ No	Access cannot be reduced
43	Can a child increase visibility of an overridden method?	✓ Yes	Access can be widened
44	Can a parent reference point to a child object?	✓ Yes	This is upcasting
45	Can a child reference point directly to a parent object?	✗ No	The object must actually be compatible with the child
46	Can downcasting be used after upcasting?	✓ Yes*	Only when the runtime object is the child type
47	Can invalid downcasting cause ClassCastException?	✓ Yes	An incompatible runtime type causes it
48	Can inheritance promote code reuse?	✓ Yes	Children can reuse accessible parent behavior
49	Can inheritance represent an is-a relationship?	✓ Yes	A child is a specialized form of its parent
50	Can every class inherit from Object directly or indirectly?	✓ Yes	Object is the root class of Java classes
51	Can a class extend Object explicitly?	✓ Yes	Object can be named as the superclass
52	Can a class extend itself?	✗ No	A class cannot be its own superclass
53	Can circular inheritance exist?	✗ No	Inheritance cycles are invalid
54	Can a constructor be inherited from Object?	✗ No	Constructors are never inherited
55	Can private fields still exist in a parent part of a child object?	✓ Yes	They are part of the parent class state but not directly accessible
56	Can inheritance be combined with encapsulation?	✓ Yes	A child can use controlled parent APIs
57	Can inheritance be combined with abstraction?	✓ Yes	Abstract parents are commonly extended
58	Can inheritance support runtime polymorphism?	✓ Yes	Overridden instance methods are dynamically dispatched
59	Can a subclass inherit accessible methods from its superclass?	✓ Yes	Inheritance provides reusable behavior

60 Can a subclass add its own fields?

✔ Yes

A child can add state

7. Encapsulation — 60 Questions

#	Question	Answer	Main Reason
1	Is encapsulation an OOP concept?	✔ Yes	It bundles data and behavior and controls access
2	Can encapsulation use private variables?	✔ Yes	private prevents direct outside access
3	Can getters support encapsulation?	✔ Yes	Getters provide controlled reading
4	Can setters support encapsulation?	✔ Yes	Setters provide controlled modification
5	Can another class directly access a private field?	✗ No	private restricts direct access
6	Can another class access a private field through a public getter?	✔ Yes	The getter provides controlled access
7	Can another class modify a private field through a public setter?	✔ Yes	The setter controls modification
8	Does encapsulation require all fields to be private?	✗ No	It is a design principle, not an absolute rule
9	Does encapsulation mean making all variables public?	✗ No	Public fields reduce access control
10	Can a setter validate data?	✔ Yes	Validation can enforce valid state
11	Can a getter return a calculated value?	✔ Yes	Getters can expose controlled behavior
12	Can a class hide implementation details?	✔ Yes	Access control can hide internal details
13	Can encapsulation improve maintainability?	✔ Yes	Internal changes can be isolated
14	Can encapsulation reduce unintended modification?	✔ Yes	Direct access can be restricted
15	Can private methods help encapsulation?	✔ Yes	They can hide internal helper behavior
16	Can a public method expose controlled access to private data?	✔ Yes	Public methods can define the class API
17	Can a constructor initialize private fields?	✔ Yes	The class can initialize its own state
18	Can a private constructor support encapsulation?	✔ Yes	It can control object creation
19	Can encapsulation be achieved without getters and setters?	✔ Yes	Other controlled methods can be used
20	Can immutable classes use encapsulation?	✔ Yes	Private final state is commonly used
21	Can a final reference field still refer to a mutable object?	✔ Yes	final prevents reassignment of the reference
22	Can a private field be static?	✔ Yes	Access modifiers and static can combine
23	Can a private field be final?	✔ Yes	final and private can combine
24	Can a private field be both static and final?	✔ Yes	This is common for constants
25	Can a setter reject invalid values?	✔ Yes	It can validate before assignment
26	Can a getter hide the actual internal representation?	✔ Yes	It can return a safe or calculated value
27	Can encapsulation help protect object invariants?	✔ Yes	Controlled updates can preserve valid state
28	Can direct public field access weaken encapsulation?	✔ Yes	It bypasses controlled access
29	Can exposing mutable internal objects weaken encapsulation?	✔ Yes	Callers may change internal state
30	Can defensive copying improve encapsulation?	✔ Yes	Copies prevent unwanted external mutation
31	Can encapsulation be used with inheritance?	✔ Yes	Parent classes can expose controlled protected/public

32	Can a subclass directly access a parent's private field?	✗ No	Private access is limited to the declaring class
33	Can a subclass use a public getter for a parent's private field?	✓ Yes	The public API can provide access
34	Can a subclass use a protected field?	✓ Yes	protected permits subclass access
35	Is protected always better than private for encapsulation?	✗ No	private generally gives stronger direct control
36	Can encapsulation hide data representation?	✓ Yes	Clients need not know internal storage
37	Can encapsulation hide implementation of operations?	✓ Yes	Public methods can hide internal steps
38	Can methods enforce business rules?	✓ Yes	Methods can validate and control changes
39	Can an object remain valid by using controlled setters?	✓ Yes	Setters can reject invalid state
40	Can a class expose only getters and no setters?	✓ Yes	This supports read-only access
41	Can a class expose methods instead of setters?	✓ Yes	Behavior methods can enforce rules
42	Can fields be private and methods public?	✓ Yes	This is a common encapsulation pattern
43	Can encapsulation reduce dependencies on internal fields?	✓ Yes	Clients use methods instead of field representation
44	Can changing a private field name break external code?	✗ Usually no	External code should use the public API
45	Can a private field be accessed using reflection?	✓ Yes*	Reflection can bypass normal access controls in permitted contexts
46	Does encapsulation provide compile-time access control?	✓ Yes	Access modifiers are enforced by Java
47	Can encapsulation improve security?	✓ Yes*	It can reduce uncontrolled access, though it is not a complete security system
48	Can an interface support encapsulation?	✓ Yes	A public contract can hide implementation
49	Can an abstract class support encapsulation?	✓ Yes	It can protect state and expose controlled methods
50	Can a class be fully encapsulated while having public methods?	✓ Yes	Public methods can be the controlled API
51	Can private constructors be used in Singleton-style designs?	✓ Yes	They restrict direct construction
52	Can immutable objects improve encapsulation?	✓ Yes	State cannot be changed after construction
53	Can encapsulation prevent every possible change to an object?	✗ No	It controls access; design determines mutability
54	Can encapsulation be achieved by access modifiers?	✓ Yes	private/protected/public help control access
55	Can encapsulation be applied to methods as well as fields?	✓ Yes	Private helpers can hide implementation
56	Can a setter return void?	✓ Yes	A setter commonly updates state without returning a value
57	Can a getter take parameters?	✓ Yes*	A method can read/calculates data with parameters, though a conventional getter usually has none
58	Can encapsulation make code easier to test?	✓ Yes*	Clear controlled interfaces can simplify testing
59	Is encapsulation the same as abstraction?	✗ No	Encapsulation controls access; abstraction hides complexity
60	Can a class expose read-only data through getters?	✓ Yes	A getter can expose data without a setter

8. Abstraction — 60 Questions

#	Question	Answer	Main Reason
1	Is abstraction an OOP concept?	✓ Yes	It focuses on essential behavior and hides details
2	Can abstraction be achieved using an abstract class?	✓ Yes	Abstract classes support partial abstraction
3	Can abstraction be achieved using an interface?	✓ Yes	Interfaces define contracts
4	Can an abstract class be instantiated directly?	✗ No	It cannot be directly instantiated
5	Can an abstract class have a constructor?	✓ Yes	Its constructor runs as part of child construction
6	Can an abstract class have concrete methods?	✓ Yes	It can contain implemented methods
7	Can an abstract class have abstract methods?	✓ Yes	Abstract methods define required behavior
8	Can an abstract class have fields?	✓ Yes	It can contain state
9	Can an abstract class have static methods?	✓ Yes	Static methods can have implementations
10	Can an abstract class have final methods?	✓ Yes	Final concrete methods are allowed
11	Can an abstract method have a body?	✗ No	An abstract method has no implementation body
12	Can an abstract method be private?	✗ No	A subclass cannot implement a private method
13	Can an abstract method be final?	✗ No	final prevents overriding
14	Can an abstract method be static?	✗ No	Static methods are not overridden
15	Can an abstract method be protected?	✓ Yes	Access modifiers are allowed where appropriate
16	Can an abstract method be public?	✓ Yes	Public abstract methods are common
17	Can an abstract class have only concrete methods?	✓ Yes*	It can be abstract even without abstract methods
18	Can an abstract class have no abstract methods?	✓ Yes	The abstract modifier can still prevent direct instantiation
19	Can a concrete class extend an abstract class?	✓ Yes	It can provide required implementations
20	Can an abstract class extend another abstract class?	✓ Yes	Abstract classes can form inheritance chains
21	Can an abstract class implement an interface?	✓ Yes	It may leave interface methods abstract
22	Can a concrete child inherit an abstract method?	✗ No*	It must implement inherited abstract methods unless another implementation exists
23	Can an abstract child leave an abstract method unimplemented?	✓ Yes	An abstract child may remain incomplete
24	Can abstraction reduce complexity?	✓ Yes	Users work with essential operations
25	Can abstraction hide implementation details?	✓ Yes	Implementation can be behind a contract
26	Can an interface have default methods?	✓ Yes	Default methods provide implementations
27	Can an interface have static methods?	✓ Yes	Static interface methods are supported
28	Can an interface have private methods?	✓ Yes	Modern Java allows private interface helper methods
29	Can an interface have constructors?	✗ No	Interfaces are not instantiated
30	Can an interface be instantiated directly?	✗ No	An interface is a type contract
31	Can an interface reference point to an implementation object?	✓ Yes	An implementing object is assignable to the interface
32	Can abstraction be achieved without an abstract class?	✓ Yes	Interfaces are another major mechanism
33	Can abstraction be achieved without an interface?	✓ Yes	Abstract classes can provide abstraction

34	Can abstract classes contain normal variables?	✓ Yes	They can maintain state
35	Can abstract classes contain static variables?	✓ Yes	Static fields are allowed
36	Can abstract classes contain private methods?	✓ Yes	Private concrete helper methods are allowed
37	Can abstract classes contain final variables?	✓ Yes	final fields are allowed
38	Can abstract classes contain synchronized methods?	✓ Yes	Normal method modifiers can be used
39	Can an abstract class have a main method?	✓ Yes	main can be static
40	Can an abstract class be final?	✗ No	final prevents subclassing while abstract requires subclass implementation
41	Can abstraction support polymorphism?	✓ Yes	Abstract references can point to concrete implementations
42	Can abstraction be used with inheritance?	✓ Yes	Abstract classes are inherited
43	Can an abstract method be overridden?	✓ Yes	A concrete subclass implements it
44	Can an abstract class override a concrete parent method and make it abstract?	✓ Yes*	A subclass can redeclare an inherited concrete method as abstract when allowed
45	Can an interface extend another interface?	✓ Yes	Interfaces support extension
46	Can an interface extend multiple interfaces?	✓ Yes	Multiple interface inheritance is supported
47	Can a class implement multiple interfaces for abstraction?	✓ Yes	It can follow multiple contracts
48	Can abstraction define what to do without exposing how?	✓ Yes	That is the central idea
49	Is abstraction the same as encapsulation?	✗ No	They solve different design problems
50	Can abstraction improve maintainability?	✓ Yes	Implementations can change behind contracts
51	Can abstraction reduce coupling?	✓ Yes	Clients depend on abstractions rather than implementations
52	Can abstraction make code reusable?	✓ Yes	Common contracts can support multiple implementations
53	Can an abstract class contain both abstract and concrete methods?	✓ Yes	This is a common partial-abstraction pattern
54	Can an interface contain constants?	✓ Yes	Interface fields are implicitly public static final
55	Can an abstract class be used as a reference type?	✓ Yes	It can reference concrete child objects
56	Can an abstract class have multiple constructors?	✓ Yes	Constructors can be overloaded
57	Can an abstract class be a parent of multiple classes?	✓ Yes	Multiple concrete children can extend it
58	Can abstraction be applied to a method API?	✓ Yes	The API can expose behavior without implementation details
59	Can an abstract class be used as a variable type?	✓ Yes	Reference variables can use abstract types
60	Can an interface reference be used as a variable type?	✓ Yes	Interfaces are valid reference types

9. Interfaces — 60 Questions

#	Question	Answer	Main Reason
1	Can we declare an interface?	✓ Yes	Interfaces define contracts
2	Can we instantiate an interface directly?	✗ No	Interfaces are not directly instantiated

3	Can an interface have abstract methods?	✓ Yes	Abstract methods define required behavior
4	Can an interface have default methods?	✓ Yes	Default methods have implementations
5	Can an interface have static methods?	✓ Yes	Static methods belong to the interface
6	Can an interface have private methods?	✓ Yes	Private helper methods are supported
7	Can an interface have constructors?	✗ No	Interfaces are not instantiated
8	Can an interface have variables?	✓ Yes	Interface fields are implicitly public static final
9	Can an interface variable be reassigned?	✗ No	Interface fields are final
10	Can a class implement an interface?	✓ Yes	Use implements
11	Can a class implement multiple interfaces?	✓ Yes	Java supports multiple interface implementation
12	Can an interface extend another interface?	✓ Yes	Use extends
13	Can an interface extend multiple interfaces?	✓ Yes	Multiple interface inheritance is allowed
14	Can an interface extend a class?	✗ No	Interfaces only extend interfaces
15	Can a class extend an interface?	✗ No	A class implements interfaces
16	Can an abstract class implement an interface?	✓ Yes	It may leave methods abstract
17	Can a concrete class implement an interface without implementing required abstract methods?	✗ No	It must implement them or remain abstract
18	Can an interface reference hold an implementing object?	✓ Yes	The object is assignable to the interface type
19	Can a default method have a body?	✓ Yes	Default methods provide implementations
20	Can an abstract interface method have a body?	✗ No	Abstract methods have no implementation
21	Can an interface static method be overridden?	✗ No	Static interface methods are not overridden
22	Can a default interface method be overridden?	✓ Yes	Implementations may provide their own version
23	Can an interface private method be overridden?	✗ No	Private methods are not inherited
24	Can an interface field be private?	✗ No	Interface fields are public static final
25	Can an interface field be protected?	✗ No	Interface fields are public
26	Can an interface field be final?	✓ Yes	All interface fields are final
27	Can an interface field be static?	✓ Yes	All interface fields are static
28	Can an interface field be public?	✓ Yes	All interface fields are public
29	Can an interface have constants?	✓ Yes	Its fields are constants
30	Can an interface contain nested classes?	✓ Yes	Nested types are allowed
31	Can an interface contain nested interfaces?	✓ Yes	Nested interfaces are allowed
32	Can an interface contain an enum?	✓ Yes	Nested enum declarations are allowed
33	Can an interface have a main method?	✓ Yes	A static main method can be declared
34	Can an interface be final?	✗ No	Interfaces are intended to be implemented/extended
35	Can an interface be abstract explicitly?	✗ No*	Interfaces are inherently abstract; explicit abstract is unnecessary and not used as a modifier
36	Can two interfaces have methods with the same signature?	✓ Yes	A class can implement the common contract
37	Can two interfaces have conflicting default methods?	✗ No*	The implementing class must resolve the conflict

38	Can a class resolve a default-method conflict?	✓ Yes	The class can override the conflicting method
39	Can an interface provide multiple inheritance?	✓ Yes	An interface can extend multiple interfaces
40	Can a class use one interface and one superclass?	✓ Yes	A class can extend one class and implement interfaces
41	Can an interface be used for abstraction?	✓ Yes	It defines a contract
42	Can an interface support runtime polymorphism?	✓ Yes	References can point to different implementations
43	Can an interface contain private static methods?	✓ Yes	Private interface helper methods can be static
44	Can an interface contain private instance methods?	✓ Yes	Private helper methods can be non-static
45	Can a default method call another interface method?	✓ Yes	It can use accessible interface methods
46	Can an interface inherit constants from parent interfaces?	✓ Yes*	Inherited fields can be accessed through the interface type
47	Can an implementing class hide an interface static method?	✗ No*	Interface static methods are not inherited as class methods
48	Can a class implement the same interface indirectly through two parents?	✓ Yes*	Repeated interface inheritance is allowed when compatible
49	Can an interface extend itself?	✗ No	Circular inheritance is invalid
50	Can interfaces form an inheritance chain?	✓ Yes	Interfaces can extend other interfaces
51	Can a functional interface have one abstract method?	✓ Yes	That is the functional-interface rule
52	Can a functional interface have default methods too?	✓ Yes	Default methods do not count as abstract methods
53	Can an interface be used as a lambda target?	✓ Yes*	It must be a functional interface
54	Can an interface have generic type parameters?	✓ Yes	Interfaces can be generic
55	Can an interface be used as a method parameter?	✓ Yes	It can define a flexible contract
56	Can an interface be used as a return type?	✓ Yes	Implementations can be returned through the interface type
57	Can an interface have fields initialized with expressions?	✓ Yes	Interface constants must be initialized
58	Can an interface have instance fields?	✗ No	Its fields are static
59	Can an interface have mutable fields?	✗ No	Its fields are final
60	Can an interface contain implementation code?	✓ Yes	Default/static/private methods can have bodies

10. Classes & Objects — 60 Questions

#	Question	Answer	Main Reason
1	Can we declare a class?	✓ Yes	Classes define types
2	Can a class have fields?	✓ Yes	Fields store state
3	Can a class have methods?	✓ Yes	Methods define behavior
4	Can a class have constructors?	✓ Yes	Constructors initialize objects
5	Can a class have multiple constructors?	✓ Yes	Constructor overloading is allowed
6	Can a class be public?	✓ Yes	A top-level class can be public
7	Can a class be private at top level?	✗ No	Top-level classes cannot be private

8	Can a nested class be private?	✓ Yes	Member classes can have access modifiers
9	Can a class be protected at top level?	✗ No	Top-level classes cannot be protected
10	Can a nested class be protected?	✓ Yes	Member classes can be protected
11	Can a class be final?	✓ Yes	final prevents inheritance
12	Can a final class be extended?	✗ No	final prevents extension
13	Can a class be abstract?	✓ Yes	Abstract classes can be incomplete
14	Can an abstract class be instantiated directly?	✗ No	Abstract classes cannot be directly instantiated
15	Can an abstract class reference point to a child object?	✓ Yes	Upcasting is allowed
16	Can one class create multiple objects?	✓ Yes	Each object has its own instance state
17	Can objects of the same class have different field values?	✓ Yes	Instance state belongs to each object
18	Can an object be created with new?	✓ Yes	new normally creates objects
19	Can an object be created without new?	✓ Yes*	Cloning/deserialization and other mechanisms can create objects
20	Can a reference variable exist without an object?	✓ Yes	It can hold null
21	Can a reference variable be null?	✓ Yes	null means no referenced object
22	Can calling an instance method through null cause an exception?	✓ Yes	It can cause NullPointerException
23	Can static members be accessed through a class name?	✓ Yes	Static members belong to the class
24	Can instance members be accessed through an object?	✓ Yes	Instance members belong to objects
25	Can a class have static variables?	✓ Yes	Static fields are class-level
26	Can a class have instance variables?	✓ Yes	Instance fields belong to objects
27	Can a class have static methods?	✓ Yes	Static methods belong to the class
28	Can a class have instance methods?	✓ Yes	Instance methods use object state
29	Can a class contain another class?	✓ Yes	Nested classes are supported
30	Can a class contain an interface?	✓ Yes	Nested interfaces are supported
31	Can a class implement an interface?	✓ Yes	Use implements
32	Can a class extend another class?	✓ Yes	Use extends
33	Can a class extend multiple classes?	✗ No	Java supports single class inheritance
34	Can a class implement multiple interfaces?	✓ Yes	Multiple interface implementation is allowed
35	Can a class have a private constructor?	✓ Yes	It can control object creation
36	Can a class have no constructor declaration?	✓ Yes	The compiler may supply a default constructor
37	Can a class have a constructor with parameters?	✓ Yes	Parameterized constructors are allowed
38	Can a class have an abstract method?	✗ No*	Only an abstract class or interface can declare abstract methods
39	Can an abstract class have an abstract method?	✓ Yes	Abstract classes may declare abstract methods
40	Can a final class have methods?	✓ Yes	final affects inheritance, not methods in general
41	Can a class have final methods?	✓ Yes	Final methods cannot be overridden
42	Can a class have private methods?	✓ Yes	Private helpers are allowed

43	Can a class have protected methods?	✓ Yes	Protected access is allowed
44	Can a class have public methods?	✓ Yes	Public methods are allowed
45	Can a class have synchronized methods?	✓ Yes	Methods may be synchronized
46	Can a class have static initialization blocks?	✓ Yes	Static blocks initialize class-level state
47	Can a class have instance initialization blocks?	✓ Yes	Instance blocks run during object construction
48	Can a class be both abstract and final?	✗ No	abstract needs extension; final prevents it
49	Can a class be nested inside a method?	✗ No*	Local classes can be declared inside methods, but they are local classes
50	Can a local class be declared inside a method?	✓ Yes	Java supports local classes
51	Can an object access private fields directly from another class?	✗ No	private restricts access
52	Can an object call public methods?	✓ Yes	Public methods are accessible
53	Can two references point to the same object?	✓ Yes	References can alias the same object
54	Can changing one alias affect the shared object?	✓ Yes	Both references point to the same object
55	Can an object become unreachable?	✓ Yes	It can have no reachable references
56	Can unreachable objects be garbage collected?	✓ Yes*	Eligible objects may be reclaimed by the JVM
57	Can a class inherit from Object?	✓ Yes	All ordinary classes inherit Object directly or indirectly
58	Can an object have its own instance variables?	✓ Yes	Each object has instance state
59	Can static variables be shared among objects?	✓ Yes	There is one class-level field
60	Can an object be passed to a method?	✓ Yes	Object references can be method arguments

11. Static, Final & Variables — 60 Questions

#	Question	Answer	Main Reason
1	Can a variable be static?	✓ Yes	Static variables belong to the class
2	Can a variable be final?	✓ Yes	final prevents reassignment
3	Can a variable be static final?	✓ Yes	This is commonly used for constants
4	Can a final variable be reassigned?	✗ No	final prevents reassignment
5	Can a static variable be accessed by class name?	✓ Yes	Static members belong to the class
6	Can an instance variable be accessed using the class name directly?	✗ No	It belongs to an object
7	Can a static variable be shared by all objects?	✓ Yes	There is one class-level copy
8	Does each object have its own instance variable?	✓ Yes	Instance state belongs to each object
9	Can a static variable be initialized in a static block?	✓ Yes	Static blocks can initialize class state
10	Can an instance variable have a default value?	✓ Yes	Fields receive default values
11	Can a local variable have an automatic default value?	✗ No	Local variables must be initialized before use
12	Can a final local variable be initialized later?	✓ Yes*	It must be definitely assigned before use
13	Can a final instance variable be initialized in a constructor?	✓ Yes	Blank final fields can be constructor-initialized

14	Can a final static variable be initialized in a static block?	✓ Yes	Blank static final fields can be initialized there
15	Can a final reference point to a mutable object?	✓ Yes	final prevents reference reassignment, not object mutation
16	Can the object referred to by a final reference be modified?	✓ Yes*	The reference is final, but the object may be mutable
17	Can a final primitive variable change value?	✗ No	Its value cannot be reassigned
18	Can a static variable change value?	✓ Yes	static does not mean constant
19	Can a static final variable change value?	✗ No	final prevents reassignment
20	Can a method be static?	✓ Yes	static methods belong to the class
21	Can a static method access an instance variable directly?	✗ No	It needs an object
22	Can a static method access a static variable directly?	✓ Yes	Both are class members
23	Can an instance method access a static variable?	✓ Yes	Instance methods can access class members
24	Can an instance method access an instance variable?	✓ Yes	It has an object context
25	Can a static method call an instance method directly?	✗ No	An object is required
26	Can an instance method call a static method?	✓ Yes	Static methods are class members
27	Can a static block access instance variables directly?	✗ No	A static block has no implicit object
28	Can an instance initialization block access static variables?	✓ Yes	Static members are available
29	Can a static block execute when the class is initialized?	✓ Yes	Static initialization occurs during class initialization
30	Can an instance initialization block execute for every object?	✓ Yes	It runs during instance construction
31	Can a variable be volatile?	✓ Yes	volatile is a variable modifier
32	Can a variable be transient?	✓ Yes	transient affects serialization of fields
33	Can a variable be abstract?	✗ No	abstract applies to classes/methods, not variables
34	Can a variable be synchronized?	✗ No	synchronized does not apply to fields
35	Can a field be private static final?	✓ Yes	All three modifiers can be combined
36	Can a constant conventionally use uppercase letters?	✓ Yes	Java naming convention uses uppercase constants
37	Can static methods be overridden?	✗ No	They are hidden
38	Can static methods be overloaded?	✓ Yes	Overloading is allowed
39	Can final methods be overridden?	✗ No	final prevents overriding
40	Can final methods be overloaded?	✓ Yes	Overloading is independent of final
41	Can a final class be inherited?	✗ No	final prevents inheritance
42	Can an abstract class be final?	✗ No	The modifiers conflict
43	Can a final reference be null?	✓ Yes*	It can be assigned null once
44	Can a blank final field be initialized in every constructor?	✓ Yes*	Each constructor path must definitely assign it
45	Can a static final field be initialized at declaration?	✓ Yes	A constant can be initialized directly
46	Can a static variable be initialized at declaration?	✓ Yes	Fields can have initializers
47	Can an instance field be initialized at declaration?	✓ Yes	Instance field initializers are supported
48	Can static variables have one value per object?	✗ No	Static fields are shared at class level

49	Can instance variables have different values per object?	✓ Yes	Each object has separate instance state
50	Can local variables be static?	✗ No	Local variables cannot use static
51	Can local variables be final?	✓ Yes	final local variables are allowed
52	Can method parameters be final?	✓ Yes	final parameters cannot be reassigned inside the method
53	Can method parameters be static?	✗ No	static does not apply to parameters
54	Can method parameters be volatile?	✗ No	volatile applies to fields
55	Can interface fields be static final implicitly?	✓ Yes	Interface fields are implicitly public static final
56	Can enum constants be final variables?	✗ No*	Enum constants are special instances, not declared as ordinary final variables
57	Can a static variable be accessed through an instance?	✓ Yes*	It is allowed but class-name access is clearer
58	Can final prevent object mutation?	✗ No	final prevents reassignment, not necessarily mutation
59	Can static mean one copy per object?	✗ No	static means class-level storage
60	Can a static final field be a constant?	✓ Yes	static final is commonly used for constants

12. Exception Handling — 60 Questions

#	Question	Answer	Main Reason
1	Can Java handle exceptions?	✓ Yes	Java provides exception-handling mechanisms
2	Can try-catch handle exceptions?	✓ Yes	catch handles matching exceptions
3	Can try be used without catch?	✓ Yes*	It can be followed by finally
4	Can try be used without finally?	✓ Yes	A try can have catch blocks
5	Can finally be used with try?	✓ Yes	finally is associated with try
6	Can finally be used without try?	✗ No	finally must follow a try
7	Can catch be used without try?	✗ No	catch must follow try
8	Can multiple catch blocks be used?	✓ Yes	Different exception types can be handled
9	Can one try have multiple catches?	✓ Yes	Multiple handlers are allowed
10	Can one catch handle multiple exception types?	✓ Yes	Multi-catch is supported
11	Can catch blocks be ordered from specific to general?	✓ Yes	Specific exceptions should come first
12	Can a general catch come before a specific catch?	✗ No	The specific catch becomes unreachable
13	Can finally execute after catch?	✓ Yes	finally normally runs after try/catch
14	Can finally execute when no exception occurs?	✓ Yes	It normally runs after successful try
15	Can finally execute when an exception is caught?	✓ Yes	It normally runs before leaving the construct
16	Can finally execute when an exception is not caught?	✓ Yes*	It normally runs before control leaves the method
17	Can finally be skipped?	✓ Yes*	Abrupt JVM termination or certain abnormal conditions can prevent it
18	Can return be used inside try?	✓ Yes	Methods can return from try

19	Can return be used inside finally?	✓ Yes*	But it can override a pending return/exception and is discouraged
20	Can throw be used to explicitly raise an exception?	✓ Yes	throw creates/propagates an exception
21	Can throws declare exceptions in a method signature?	✓ Yes	throws declares possible checked exceptions
22	Are throw and throws the same?	✗ No	throw raises an exception; throws declares exceptions
23	Can a method throw a checked exception?	✓ Yes	It must handle or declare it
24	Can a method throw an unchecked exception?	✓ Yes	Unchecked exceptions need not be declared
25	Can a method declare multiple exceptions with throws?	✓ Yes	Multiple exception types can be declared
26	Can a catch block catch Exception?	✓ Yes	Exception is a broad exception type
27	Can a catch block catch Throwable?	✓ Yes	Throwable is the root of exceptions/errors
28	Should Error normally be caught?	✗ No*	Errors generally indicate serious JVM/system problems
29	Is NullPointerException unchecked?	✓ Yes	It extends RuntimeException
30	Is ArithmeticException unchecked?	✓ Yes	It extends RuntimeException
31	Is IOException checked?	✓ Yes	It is a checked exception
32	Is SQLException checked?	✓ Yes	It is a checked exception
33	Is RuntimeException checked?	✗ No	RuntimeException is unchecked
34	Is Exception always checked?	✗ No*	RuntimeException subclasses are unchecked
35	Can custom exceptions be created?	✓ Yes	Classes can extend Exception or RuntimeException
36	Can a custom checked exception extend Exception?	✓ Yes	Exception subclasses are checked unless RuntimeException
37	Can a custom unchecked exception extend RuntimeException?	✓ Yes	RuntimeException subclasses are unchecked
38	Can an exception be rethrown?	✓ Yes	A caught exception can be thrown again
39	Can an exception be wrapped in another exception?	✓ Yes	Constructors can use a cause
40	Can getCause retrieve the underlying cause?	✓ Yes	Throwable stores a cause
41	Can try-with-resources close resources automatically?	✓ Yes	It closes AutoCloseable resources
42	Can try-with-resources use multiple resources?	✓ Yes	Multiple resources can be declared
43	Must a resource implement AutoCloseable for try-with-resources?	✓ Yes	The resource must be AutoCloseable
44	Can finally be replaced by try-with-resources for resource cleanup?	✓ Yes*	Try-with-resources is preferred for AutoCloseable resources
45	Can an exception be thrown from a constructor?	✓ Yes	Constructors can throw exceptions
46	Can an exception be thrown from a static method?	✓ Yes	Static methods can throw exceptions
47	Can an exception be thrown from an instance method?	✓ Yes	Instance methods can throw exceptions
48	Can overriding method throw broader checked exceptions?	✗ No	Checked exceptions cannot be broadened
49	Can overriding method throw narrower checked exceptions?	✓ Yes	The child may reduce checked exceptions
50	Can overriding method add unchecked exceptions?	✓ Yes	Unchecked exceptions are not restricted
51	Can a catch block throw another exception?	✓ Yes	It can throw or rethrow
52	Can finally throw an exception?	✓ Yes	It may throw, though this can hide another exception

53	Can a try block contain nested try blocks?	✓ Yes	Try blocks can be nested
54	Can catch blocks be nested?	✓ Yes*	Nested try-catch structures are allowed
55	Can exceptions propagate up the call stack?	✓ Yes	Uncaught exceptions move to callers
56	Can main declare throws Exception?	✓ Yes	main can declare exceptions
57	Can Java automatically handle every exception?	✗ No	Unhandled exceptions can terminate the thread
58	Can an exception object contain a message?	✓ Yes	Throwable supports exception messages
59	Can an exception be caught and handled in a caller method?	✓ Yes	Exceptions can propagate up the call stack
60	Can an exception be caught using its parent exception type?	✓ Yes	A parent exception can catch compatible subclasses

13. Strings & Arrays — 60 Questions

#	Question	Answer	Main Reason
1	Is String a class in Java?	✓ Yes	String is a class in java.lang
2	Is String immutable?	✓ Yes	A String object's value cannot be changed
3	Can a String reference be changed?	✓ Yes	The reference can point to another String
4	Can a String be created using a literal?	✓ Yes	String literals are supported
5	Can a String be created using new?	✓ Yes	new String(...) creates an object
6	Can two equal string literals share the same pooled object?	✓ Yes*	String literals are interned in the string pool
7	Can == compare String contents reliably?	✗ No	== compares references
8	Can equals compare String contents?	✓ Yes	String overrides equals for content comparison
9	Can String be concatenated with +?	✓ Yes	The + operator supports string concatenation
10	Can String contain spaces?	✓ Yes	Spaces are valid characters
11	Can String contain digits?	✓ Yes	Digits can be characters
12	Can String be converted to char array?	✓ Yes	toCharArray() does this
13	Can char array be converted to String?	✓ Yes	String constructors can use char arrays
14	Can String length be found with length()?	✓ Yes	length() returns character count
15	Can a String be changed using replace?	✗ No*	replace returns a new String
16	Can substring return part of a String?	✓ Yes	substring creates/returns a String
17	Can StringBuilder be mutable?	✓ Yes	Its contents can change
18	Is StringBuilder a String?	✗ No	It is a separate mutable class
19	Is StringBuffer mutable?	✓ Yes	It is a mutable character sequence
20	Can StringBuilder be faster than String concatenation in loops?	✓ Yes*	It can avoid repeated String object creation
21	Can an array store multiple values?	✓ Yes	Arrays hold multiple elements
22	Can an array store primitive values?	✓ Yes	Primitive arrays are supported
23	Can an array store object references?	✓ Yes	Reference arrays are supported
24	Can an array store different primitive types together?	✗ No	An array has one component type

25	Can an Object[] store different object types?	✔ Yes*	Objects of compatible reference types can be stored
26	Can an int[] store Integer objects directly?	✘ No	int[] stores primitive int values
27	Can an Integer[] store Integer objects?	✔ Yes	Integer[] stores Integer references
28	Can an array size change after creation?	✘ No	Array length is fixed
29	Can an array be resized using a new array?	✔ Yes	A new array can have a different length
30	Can an array have length zero?	✔ Yes	Zero-length arrays are valid
31	Can an array index start at 0?	✔ Yes	Java arrays are zero-indexed
32	Can an array index be negative?	✘ No	Valid indices start at zero
33	Can accessing an invalid array index throw an exception?	✔ Yes	ArrayIndexOutOfBoundsException can occur
34	Can array length be obtained using length?	✔ Yes	length is an array field
35	Is array length a method?	✘ No	length is a field, unlike String.length()
36	Can a multidimensional array be declared?	✔ Yes	Java supports arrays of arrays
37	Can rows in a multidimensional array have different lengths?	✔ Yes	Java supports jagged arrays
38	Can an array be initialized at declaration?	✔ Yes	Array initializer syntax is supported
39	Can an array be initialized with new?	✔ Yes	new allocates the array
40	Can an array reference be null?	✔ Yes	Array variables are references
41	Can accessing a null array cause NullPointerException?	✔ Yes	There is no array object to access
42	Can an array contain null?	✔ Yes*	Reference arrays can contain null
43	Can a primitive array contain null?	✘ No	Primitive elements cannot be null
44	Can an object array contain null?	✔ Yes	References can be null
45	Can arrays be passed to methods?	✔ Yes	Arrays are reference types
46	Can methods return arrays?	✔ Yes	Array types can be return types
47	Can arrays be copied using System.arraycopy?	✔ Yes	Java provides array-copy support
48	Can arrays be sorted using Arrays.sort?	✔ Yes	java.util.Arrays provides sorting
49	Can Arrays.equals compare array contents?	✔ Yes	It compares elements
50	Can == compare array contents?	✘ No	== compares array references
51	Can enhanced for loop iterate over an array?	✔ Yes	The enhanced for loop supports arrays
52	Can an array store another array?	✔ Yes	An array can contain array references
53	Can an array be an object in Java?	✔ Yes	Arrays are objects
54	Can array elements have default values?	✔ Yes	Array elements receive default values
55	Can int array elements default to zero?	✔ Yes	Numeric primitive defaults are zero
56	Can boolean array elements default to false?	✔ Yes	boolean default is false
57	Can reference array elements default to null?	✔ Yes	Reference default is null
58	Can an array declaration include a fixed length before new?	✘ No	Length is specified in the creation expression
59	Can arrays support generics directly with primitive types?	✘ No	Generics require reference types
60	Can Integer[] be used where Number[] is expected?	✔ Yes*	Arrays are covariant for reference types

★ QUICK OOP REVISION

Concept	Simple Meaning	Main Feature
Encapsulation	Binding data and methods + controlling access	private, getters, setters
Inheritance	Acquiring properties/behavior from another class	extends
Polymorphism	One name, different behavior	Overloading / Overriding
Abstraction	Hiding implementation details	abstract / interface

★ IMPORTANT INTERVIEW “NO” ANSWERS

- ❌ Static methods cannot be overridden.
- ❌ Private methods cannot be overridden.
- ❌ Final methods cannot be overridden.
- ❌ Constructors cannot be overridden, static, final, or abstract.
- ❌ Return type alone cannot overload a method.
- ❌ A class cannot extend multiple classes.
- ❌ An interface cannot extend a class.
- ❌ An interface cannot have a constructor.
- ❌ An abstract class cannot be directly instantiated.
- ❌ A final class cannot be inherited.
- ❌ An abstract method cannot be static, final, or private.
- ❌ An interface field cannot be reassigned.

★ IMPORTANT INTERVIEW “YES” ANSWERS

- ✅ Static methods can be overloaded.
- ✅ Non-static methods can be overloaded.
- ✅ Constructors can be overloaded.
- ✅ Abstract methods can be overridden.
- ✅ Static and non-static methods can be overloaded when parameter lists differ.
- ✅ A class can implement multiple interfaces.
- ✅ An interface can extend multiple interfaces.
- ✅ An abstract class can have constructors and concrete methods.
- ✅ Interfaces can have default, static, and private methods.
- ✅ A parent reference can hold a child object.